BDA Lab Manual, Part-2

Program 5:

Implement Functions: Count – Sort – Limit – Skip – Aggregate using MongoDB

Introduction to MongoDB

MongoDB is

1. Cross-platform.
2. Open source.
3. Non-relational.
4. Distributed.
5. NoSQL.
6. Document-oriented data store.

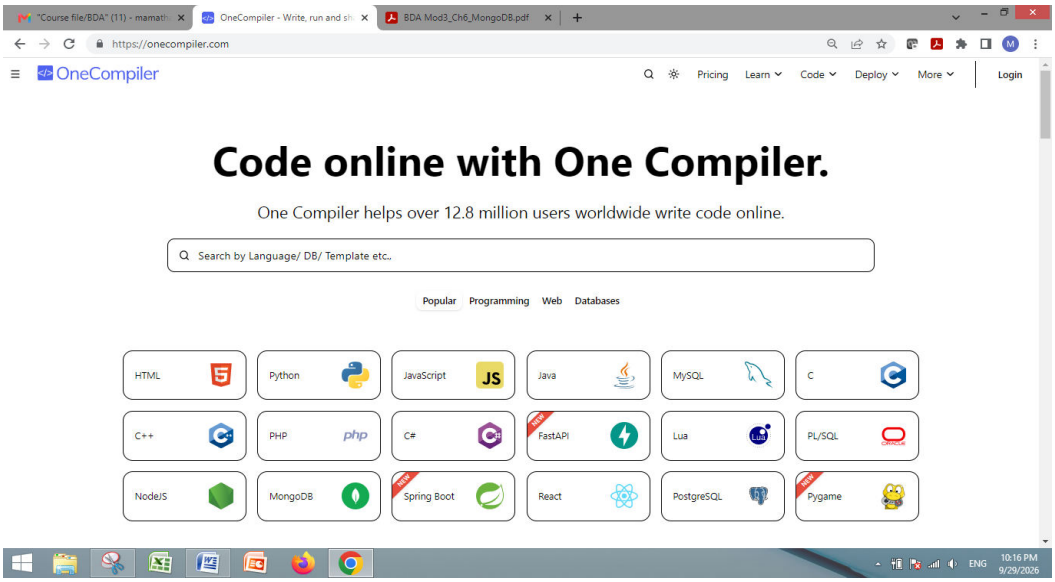
RDBMS	MongoDB
Database	Database
Table	Collection
Record	Document
Columns	Fields / Key Value pairs
Index	Index
Joins	Embedded documents
Primary Key	Primary key (_id is a identifier)

Install MongoDB

- **Mongo Shell** (mongosh) and MongoDB Server (Windows - OS)

or

- use online compiler **– OneCompiler**



Create and manipulate database using MongoDB CRUD operations

CRUD (Create, Read Update, and Delete) operations in MongoDB

Create → Creation of data is done using insert() or update() or save() method.
Read → Reading the data is performed using the find() method.
Update → Update to data is accomplished using the update() method with UPSERT set to false.
Delete → a document is Deleted using the remove() method.

Function: Count

Objective: To find the number of documents in the Students collection.

Act:

```
db.Students.count()
```

Function: Sort

Objective: To sort the documents from the Students collection in the ascending order of StudName.

Act:

```
db.Students.find().sort({StudName:1}).pretty();
```

Function: Limit

Objective: To retrieve the first 3 documents from the Students collection wherein the Grade is VII.

Act:

```
db.Students.find({Grade:"VII"}).limit(3).pretty();
```

Functions: Skip

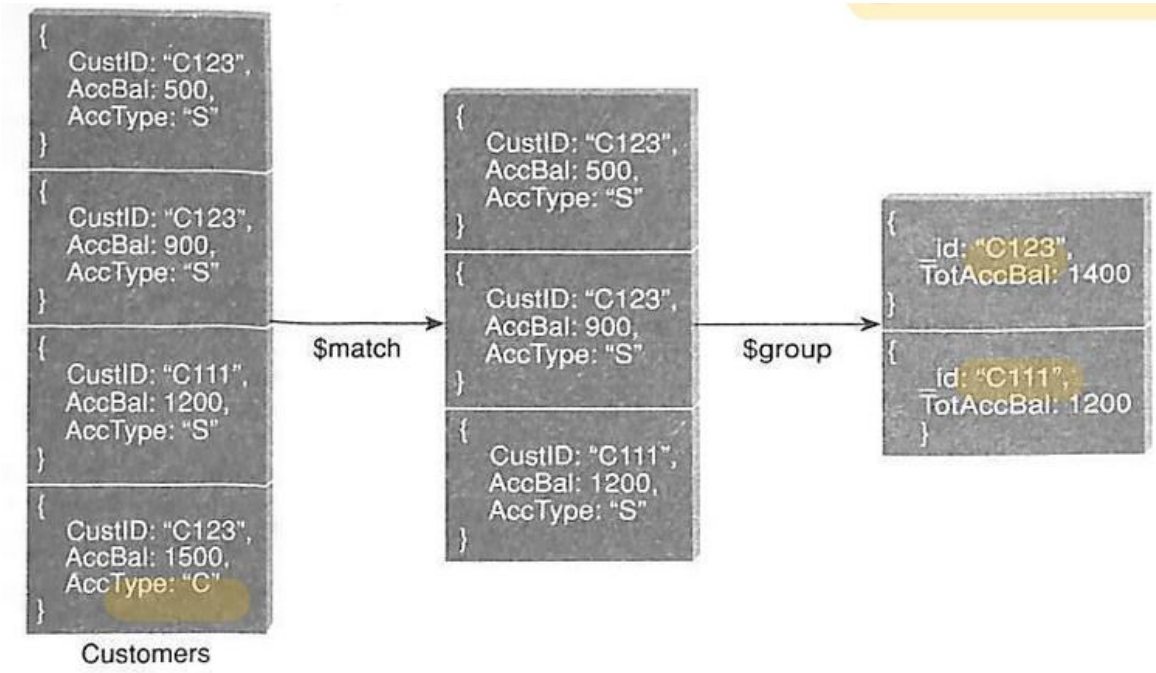
Objective: To skip the first 2 documents from the Students collection.

Act:

```
db.Students.find().skip(2).pretty();
```

MongoDB to Implement, Aggregate function

```
db.customers.insertMany([
  {custid: "c132", accbalance: 500, acctype: "SB"},
  {custid: "c132", accbalance: 600, acctype: "SB"},
  {custid: "c111", accbalance: 800, acctype: "SB"},
  {custid: "c132", accbalance: 700, acctype: "RD"}
]);
db.customers.find().pretty();
```



To group on "CustID" and compute the sum of "AccBal",

```
db.customers.aggregate({ $group: {_id: "$custid", totaccbal: {$sum: "$accbalance"}}});
```

In order to first filter on "AccType:S" and then group it on "CustID" and then compute the sum of "AccBal", use the below syntax:

```
db.customers.aggregate({ $match: {acctype:"SB"}},
{ $group: {_id: "$custid", totaccbal: {$sum: "$accbalance"}}});
```

To group on "CustID" and compute the average of the "AccBal" for each group:

```
db.customers.aggregate({ $group: {_id: "$custid", totaccbal: {$avg: "$accbalance"}}});
```

To group on "CustID" and determine the maximum "AccBal" for each group:

```
db.customers.aggregate({ $group: {_id: "$custid", totaccbal: {$max: "$accbalance"}}});
```

To group on "CustID" and determine the minimum "AccBal" for each group:

```
db.customers.aggregate({ $group: {_id: "$custid", totaccbal: {$min: "$accbalance"}}});
```

Program 5: MongoDB Query Language (MQL), to Implement Functions: Count – Sort – Limit – Skip

Step 1: Create a database and collection

```
use bda_lab

db.students.insertMany([
  {name: "Anu", dept: "ISE", marks: 85},
  {name: "Ravi", dept: "CSE", marks: 72},
  {name: "Meena", dept: "ISE", marks: 92},
  {name: "Kiran", dept: "ECE", marks: 65},
  {name: "Priya", dept: "CSE", marks: 88},
  {name: "Arun", dept: "ISE", marks: 78}
])
```

Step 2: Count Function, Count all students.

```
db.students.countDocuments()
```

Output: 6

Count students in ISE.

```
db.students.countDocuments({dept: "ISE"})
```

Output: 3

Step 3: Sort function, Sort students by marks in descending order.

```
db.students.find().sort({marks: -1})
```

Output:
Meena 92
Priya 88
Anu 85
Arun 78
Ravi 72
Kiran 65

Step 4: Limit function, Display the top three students.

```
db.students.find().sort({marks: -1}).limit(3)
```

Output:
Meena 92
Priya 88
Anu 85

Step 5: Skip function, Skip the first two records after sorting.

```
db.students.find().sort({marks: -1}).skip(2)
```

Output:

```
Anu    85
Arun   78
Ravi   72
Kiran  65
```

Step 6: Aggregate function, Calculate average marks and student count for each department.

```
db.students.aggregate([
  {
    $group: {
      _id: "$dept",
      averageMarks: {$avg: "$marks"},
      totalStudents: {$sum: 1}
    }
  },
  {
    $sort: {_id: 1}
  }
])
```

Output:

Department	Average marks	Students
CSE	80	2
ECE	65	1
ISE	85	3

Program 6 : Write Pig Latin scripts to sort, group, join, project, and filter the data.

- Apache Pig is a platform for data analysis. It is an alternative to MapReduce Programming.
- It provides an **engine** for executing **data flows** (how your data should flow). Pig processes data in parallel on the Hadoop cluster.
- It provides a language called "**Pig Latin**" to express data flows.
- The main components of Pig are as follows:
  - Data flow language (**Pig Latin**).
  - Interactive shell where you can type Pig Latin statements (**Grunt**).
  - Pig interpreter and execution engine.
- You can run Pig in two ways:
  - Interactive Mode.
  - Batch Mode.

1. Pig Latin statements are basic constructs to process data using Pig.
2. Pig Latin statement is an operator.
3. An operator in Pig Latin takes a relation as input and yields another relation as output.
4. Pig Latin statements include schemas and expressions to process data.
5. Pig Latin statements should end with a semi-colon.

Pig Latin Statements are generally ordered as follows:

1. **LOAD** statement that reads data from the file system.
2. Series of statements to perform transformations.
3. **DUMP** or **STORE** to display/store result.

The following is a simple Pig Latin script to load, filter, and store "student" data.

```
A = load 'student' (rollno, name, gpa);
A = filter A by gpa > 4.0;
A = foreach A generate UPPER (name);
STORE A INTO 'myreport'
```

Start the Software: Apache Hadoop and Apache Pig.

Step 1: Create the input file (gedit students.csv)

Create a file named students.csv with the following contents:

[cloudera@quickstart ~]$ gedit stuents.csv

```
1,Anu,ISE,85
2,Ravi,CSE,72
3,Meena,ISE,92
4,Kiran,ECE,65
5,Priya,CSE,88
6,Arun,ISE,78
```

In the Cloudera terminal, save the file in home directory under root

[cloudera@quickstart ~]$ hdfs dfs -mkdir -p /pigdata

[cloudera@quickstart ~]$ hdfs dfs -put students.csv/pigdata

[cloudera@quickstart ~]$ hdfs dfs -cat /pigdata/students.csv

Step 2: Start Pig

to enter, grunt shell - Pig environment

[cloudera@quickstart ~]$ sudo pig

The Pig prompt appear as: **grunt>**

Step 3: Load the dataset

```
students = LOAD 'students.csv'
USING PigStorage(',')
AS (id:int, name:chararray,
   dept:chararray, marks:int);
```

- Display the dataset:

```
DUMP students;
```

Step 4: Sort

- Sort students by marks in descending order.

```
sorted_students = ORDER students BY marks DESC;
DUMP sorted_students;
```

Output:

(3,Meena,ISE,92)

(5,Priya,CSE,88)

(1,Anu,ISE,85)

Step 5: Group

- Group students by department.

```
grouped_students = GROUP students BY dept;
DUMP grouped_students;
```

- Count students in each department.

```
dept_count = FOREACH grouped_students
GENERATE group AS department,
COUNT(students) AS total_students;
DUMP dept_count;
```

Output: (CSE,2)
(ECE,1)
(ISE,3)

Step 6: Join

- Create a second file named departments.csv.

```
CSE,Computer Science
ISE,Information Science
ECE,Electronics
```

- Load the file:

```
departments = LOAD 'departments.csv'
USING PigStorage(',')
AS (dept_code:chararray,
   dept_name:chararray);
```

- Join both datasets:

```
joined_data = JOIN students BY dept,
departments BY dept_code;
DUMP joined_data;
```

- Display selected fields:

```
join_result = FOREACH joined_data
GENERATE students::name,
departments::dept_name,
students::marks;
```

DUMP join_result;

Output:

(Anu,Information Science,85)

(Ravi,Computer Science,72)

(Meena,Information Science,92)

Step 7: Projection

- Select only student names and marks.

projected_data = FOREACH students

GENERATE name, marks;

DUMP projected_data;

Output:

(Anu,85)

(Ravi,72)

(Meena,92)

Step 8: Filter

- Display students who scored more than 80 marks.

filtered_students = FILTER students BY marks > 80;

DUMP filtered_students;

Output:

(1,Anu,ISE,85)

(3,Meena,ISE,92)

(5,Priya,CSE,88)

Step 9: Store output

STORE filtered_students

INTO 'filtered_output'

USING PigStorage(',');

Program 7: Use Hive to create, alter, and drop databases, tables, views, functions, and indexes.

- Hive is a Data Warehousing tool.
- Hive is used to query structured data built on top of Hadoop.
- Hive provides HQL (Hive Query Language) which is similar to SQL.
- A Hive database contains several tables. Each table is constituted of rows and columns. In Hive, tables are stored as a folder and partition tables are stored as a sub-directory.
- Bucketed tables are stored as a file.

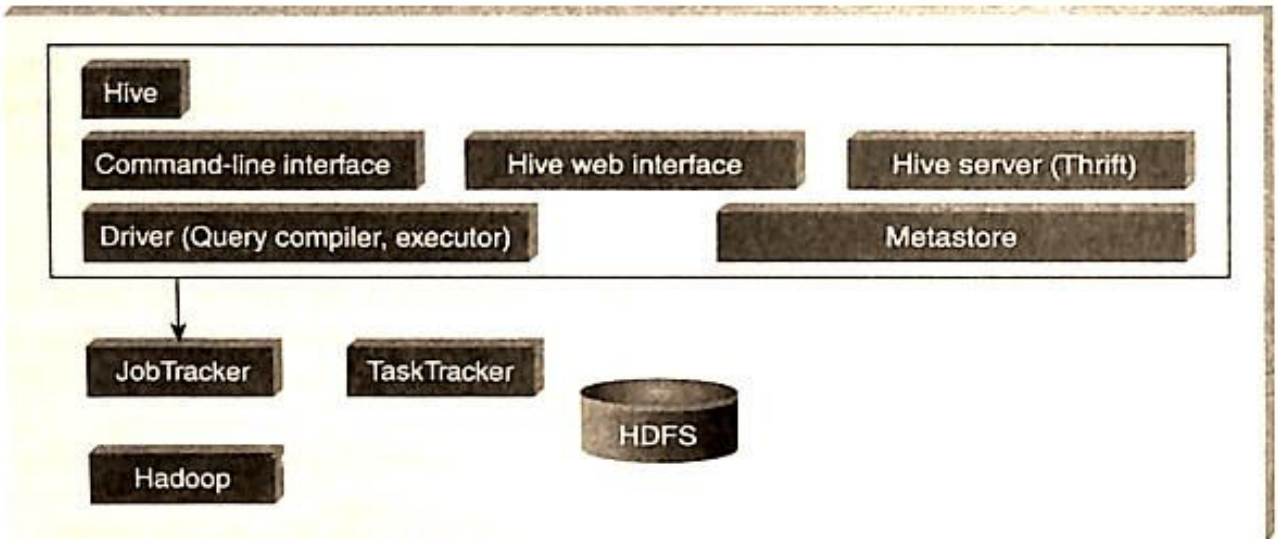


fig. Hive Architecture

Features	Pig	Hive
Used By	Programmers and Researchers	Analyst
Used For	Programming	Reporting
Language	Procedural data flow language	SQL Like
Suitable For	Semi - Structured	Structured
Schema/Types	Explicit	Implicit
UDF Support	YES	YES
Join/Order/Sort	YES	YES
DFS Direct Access	YES (Implicit)	YES (Explicit)
Web Interface	YES	NO
Partitions	YES	NO
Shell	YES	YES

Software required: Apache Hadoop and Hive.

Step 1: Start Hive

Open the Cloudera terminal and type: [cloudera@quickstart ~]$ **sudo hive**

The prompt appear as: **hive>**

Step 2: Database operations

- Create a database

```
CREATE DATABASE bda_lab;
```

- Display databases

```
SHOW DATABASES;
```

- Alter database properties

```
ALTER DATABASE bda_lab
SET DBPROPERTIES ('purpose'='BDA Lab');
```

- Select the database

```
USE bda_lab;
```

- Drop the database

```
DROP DATABASE bda_lab;
```

(Run the drop command after completing the experiments. If the database contains tables, remove them first or use CASCADE.)

Step 3: Table operations

- Create a database and select it:

```
CREATE DATABASE IF NOT EXISTS bda_lab;
USE bda_lab;
```

- Create a table

```
CREATE TABLE students (
    id INT,
    name STRING,
    dept STRING,
    marks INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

- Display table structure

```
DESCRIBE students;
```

- Insert records

```
INSERT INTO TABLE students VALUES
(1,'Anu','ISE',85),
(2,'Ravi','CSE',72),
(3,'Meena','ISE',92),
(4,'Kiran','ECE',65),
(5,'Priya','CSE',88);
```

- Display the records

```
SELECT * FROM students;
```

- Alter the table

```
ALTER TABLE students
ADD COLUMNS (email STRING);
```

- Rename the table

```
ALTER TABLE students
RENAME TO students_details;
```

- Drop the table

```
DROP TABLE students_details;
```

Step 4: View operations, Recreate the students table and insert the sample records before executing these commands.

- Create a view

```
CREATE VIEW high_scorers AS
SELECT name, dept, marks
FROM students
WHERE marks >= 80;
```

- Display the view

```
SELECT * FROM high_scorers;
```

- Alter the view

```
ALTER VIEW high_scorers AS
```

```
SELECT name, marks
FROM students
WHERE marks >= 85;
```

- Drop the view

```
DROP VIEW high_scorers;
```

Step 5: Function operations

- Display available functions

```
SHOW FUNCTIONS;
```

- Use a built-in function

```
SELECT UPPER('big data analytics');
```

output: BIG DATA ANALYTICS

- Create a temporary User-Defined Function (UDF)

```
CREATE TEMPORARY FUNCTION my_function
AS 'com.example.MyFunction';
```

- Drop the temporary function

```
DROP TEMPORARY FUNCTION my_function;
```

Step 6: Index operations, For older Hive versions that support traditional indexes:

- Create an index

```
CREATE INDEX student_dept_idx
ON TABLE students (dept)
AS 'COMPACT'
WITH DEFERRED REBUILD;
```

- Rebuild the index

```
ALTER INDEX student_dept_idx
ON students REBUILD;
```

- Display indexes

```
SHOW INDEXES ON students;
```

- Drop the index

```
DROP INDEX student_dept_idx ON students;
```

Program 8: Spark program to count words and characters

Spark processes large datasets using distributed processing techniques

To execute the program in Google Colab or Jupyter Notebook

Google Colab allows running PySpark in local mode without installing Hadoop or configuring a separate cluster. We need to install PySpark, prepare a text file, and execute the code in notebook cells. (for more details, refer **Appendix -8**)

Step 1: Open Google Colab

1. Open Google Colab.
2. Sign in using your Google account.
3. Click new notebook.
4. Rename the notebook to PySpark_WordCount.

Step 2: Install PySpark

In the first code cell, enter the following pip command and click the Run button.

```
%pip install -q pyspark
```

Step 3: Create the input text file

In a new code cell, enter this code. Creates a sample text file named input.txt in Colab's /content directory.

```
# Create a sample input file

text = """Spark is fast
Spark is powerful
Big data uses Spark
Big data is useful
"""

with open("/content/input.txt", "w") as file:
    file.write(text)

print("Input file created successfully!")
print(text)
```

Create a sample input file

```
text = """Spark is fast
Spark is powerful
Big data uses Spark
Big data is useful """
with open("/content/input.txt", "w") as file:
    file.write(text)
print("Input file created successfully!")
print(text)
```

OUTPUT: Input file created successfully!
Spark is fast
Spark is powerful
Big data uses Spark
Big data is useful

PySpark program to count words and characters

```
import re
from pyspark.sql import SparkSession

# Create Spark session
spark = (
    SparkSession.builder
    .master("local[*]")
    .appName("SparkWordCount")
    .getOrCreate()
)

sc = spark.sparkContext

# Set the minimum word occurrence threshold
threshold = 2

# Read the input file and split lines into lowercase words
tokenized = sc.textFile("/content/input.txt").flatMap(
    lambda line: re.findall(r"\b\w+\b", line.lower())
)
```

```
# Count occurrences of each word
wordCounts = tokenized.map(
    lambda word: (word, 1)
).reduceByKey(
    lambda v1, v2: v1 + v2
)

# Display word counts
print("Word counts:")
for word, count in sorted(wordCounts.collect()):
    print(word, count)

# Filter words occurring at least twice
filtered = wordCounts.filter(
    lambda pair: pair[1] >= threshold
)
```

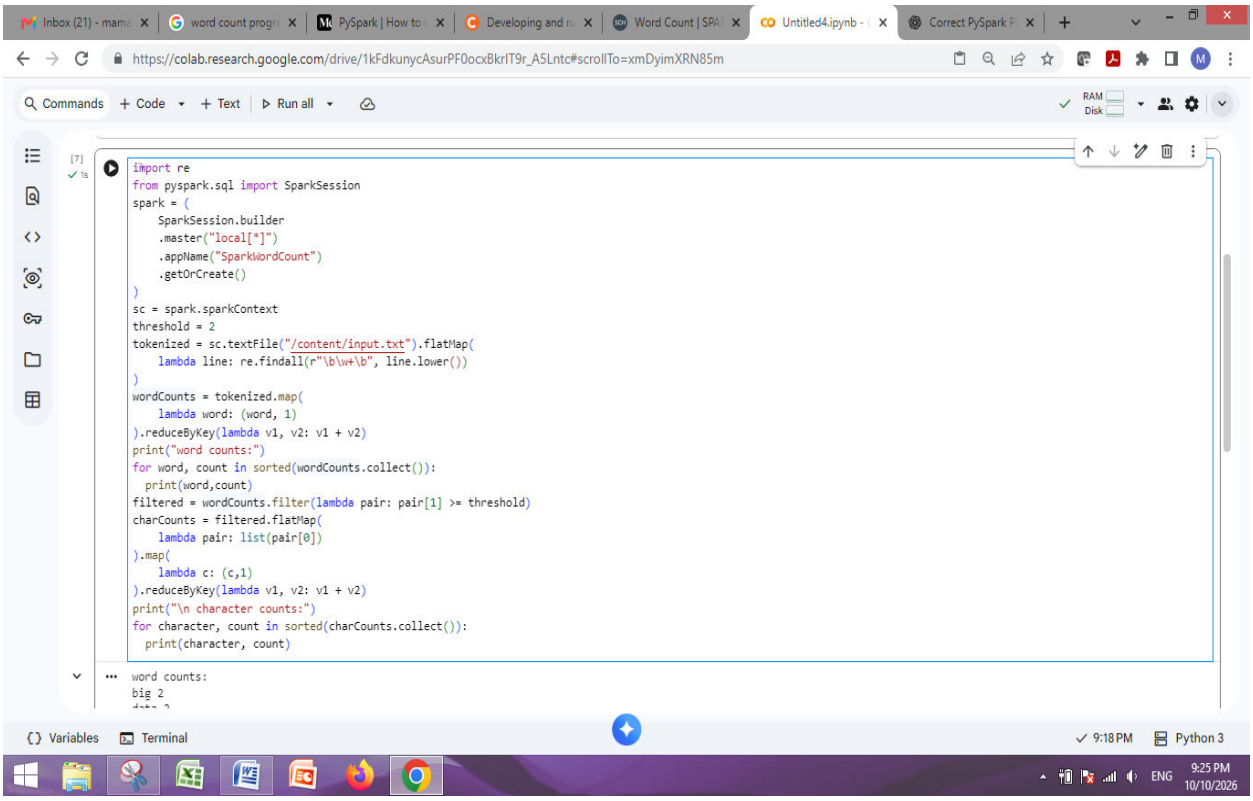
```
# Count characters in the qualifying words
charCounts = filtered.flatMap(
    lambda pair: list(pair[0])
).map(
    lambda c: (c, 1)
).reduceByKey(
    lambda v1, v2: v1 + v2
)

# Display final character counts
print("\nCharacter counts:")
for character, count in sorted(charCounts.collect()):
    print(character, count)
```

Step 4: Execute the PySpark program

Create another code cell and run the following program. This uses SparkSession, which provides access to the Spark context.

Word-count PySpark program execution in co-lab




```
import re
from pyspark.sql import SparkSession

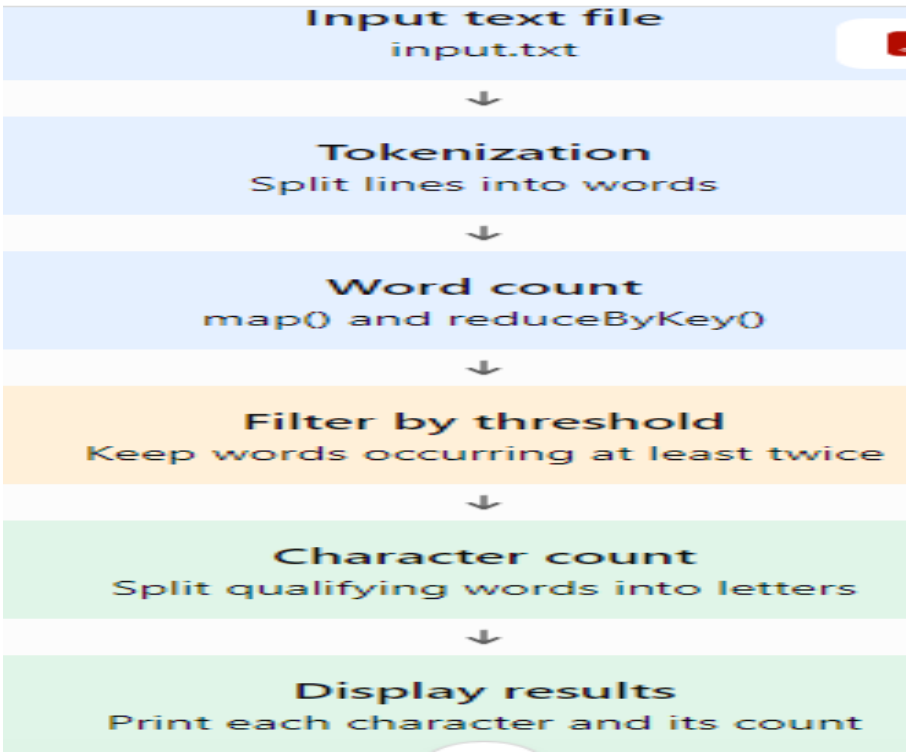
# Create Spark session
spark = (
    SparkSession.builder
    .master("local[*]")
    .appName("SparkWordCount")
    .getOrCreate()
)
sc = spark.sparkContext
# Set the minimum word occurrence threshold
threshold = 2
# Read the input file and split lines into lowercase words
tokenized = sc.textFile("/content/input.txt").flatMap(
    lambda line: re.findall(r"\b\w+\b", line.lower())
)
# Count occurrences of each word
wordCounts = tokenized.map(
    lambda word: (word, 1)
).reduceByKey(
    lambda v1, v2: v1 + v2
)
# Display word counts
print("Word counts:")
for word, count in sorted(wordCounts.collect()):
    print(word, count)
```

```
# Filter words occurring at least twice
filtered = wordCounts.filter(
    lambda pair: pair[1] >= threshold
)
# Count characters in the qualifying words
charCounts = filtered.flatMap(
    lambda pair: list(pair[0])
).map(
    lambda c: (c, 1)
).reduceByKey(
    lambda v1, v2: v1 + v2
)
# Display final character counts
print("\nCharacter counts:")
for character, count in sorted(charCounts.collect()):
    print(character, count)
```

OUTPUT:

```
Word counts:
big 2
data 2
fast 1
is 3
powerful 1
spark 3
```

Program flow



Program 9: Data Analysis and Report Generation Using CDH and HUE

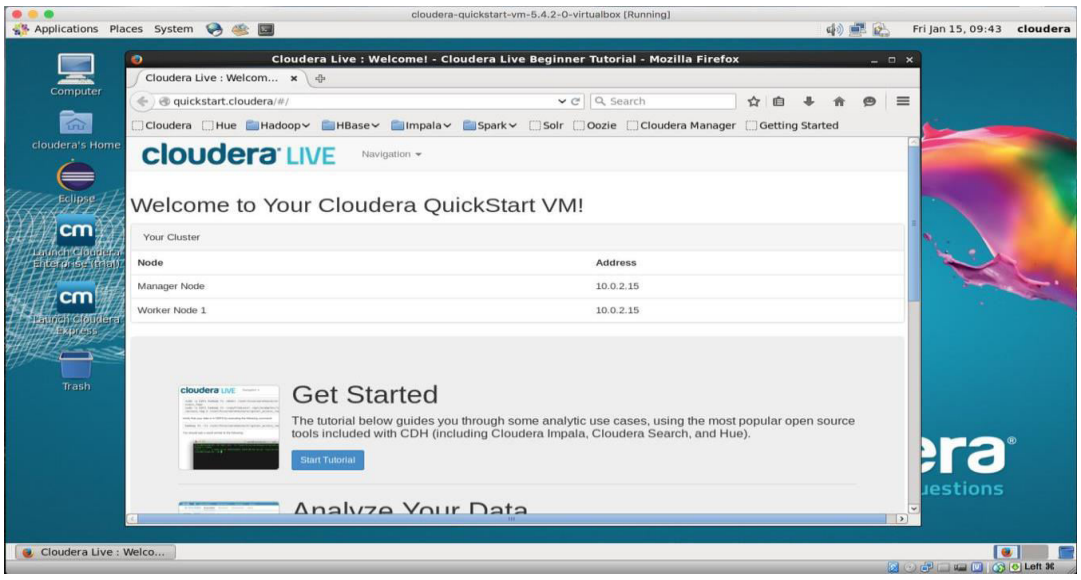
Aim: use CDH (Cloudera Distribution for Hadoop) and HUE (Hadoop User Interface) to upload, store, analyze, and generate reports from a sample dataset using Hadoop Distributed File System (HDFS) and Hive queries.

CDH is a Hadoop distribution that provides components for storing and processing large datasets, such as HDFS, YARN, MapReduce, and Hive.

HUE provides a web-based interface for browsing HDFS files, exploring tables, running SQL queries through configured Hive or Impala services, and exporting query results.

Process steps of program9 is as follows, (refer **Appendix – 9** for details)

1. The relationship is simple:
2. Sample dataset
3. CSV file on your computer
4. HUE File Browser
5. Upload the CSV to HDFS
6. Hive table and SQL queries
7. Analyze and summarize the data
8. Report generation
9. Query results, CSV export and report



- Start VirtualBox
- start, Cloudera QuickStart VM.
- Select Hadoop, -> open all tabs (run Name node, Secondary Namenode, etc)
- select Namenode -> utilities -> Browse file system
- Open VM terminal, type following commands

[cloudera@quickstart ~]$ hostname

[cloudera@quickstart ~]$ hdfs dfs -ls /

If HDFS responds with directory information, that is a useful sign that HDFS is accessible

- **Create two .CSV input files (use gedit)**

[cloudera@quickstart ~]$ gedit sales.csv

/*order_id,customer_name,product,quantity,price */

1,Alice,Laptop,1,1200

2,Bob,Mouse,2,25

3,Charlie,Keyboard,1,75

4,David,Laptop,1,1150

5,Eve,Monitor,2,300

6,Frank,Mouse,1,25sales.csv

7,Grace,Keyboard,2,75

8,Hank,Monitor,1,300

[cloudera@quickstart ~]$ gedit cust.csv

/*customer_name,location */

Alice,New York

Bob,California

Charlie,Texas

David,Florida

Eve,Texas

Frank,New York

Grace,California

Hank,Florida

- **Thru Cloudera terminal, upload the input files** in 'user' directory under root

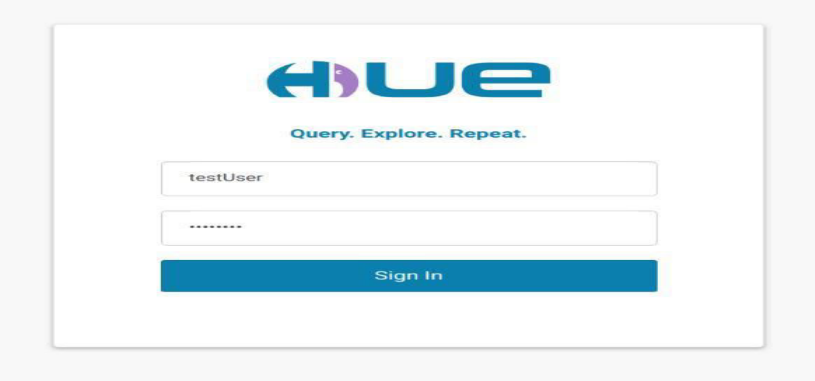
[cloudera@quickstart ~]$ hdfs dfs -mkdir -p /user/cloudera/sales_data

[cloudera@quickstart ~]$ hdfs dfs -put cust.csv /user/cloudera/sales_data/

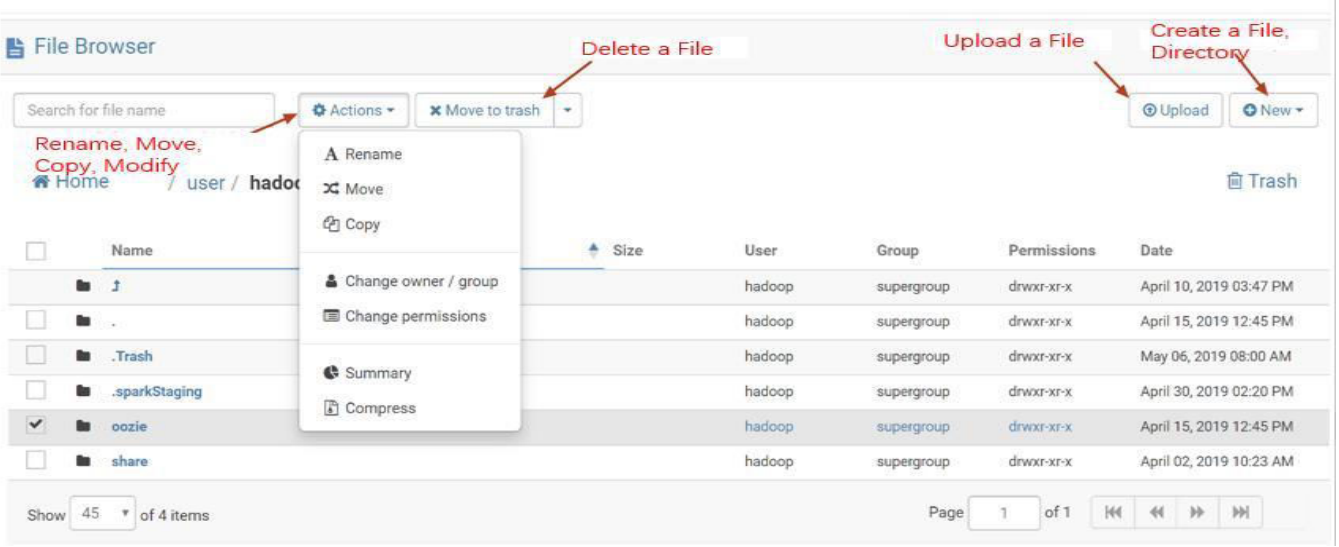
[cloudera@quickstart ~]$ hdfs dfs -put sales.csv /user/cloudera/sales_data/

[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/sales_data/

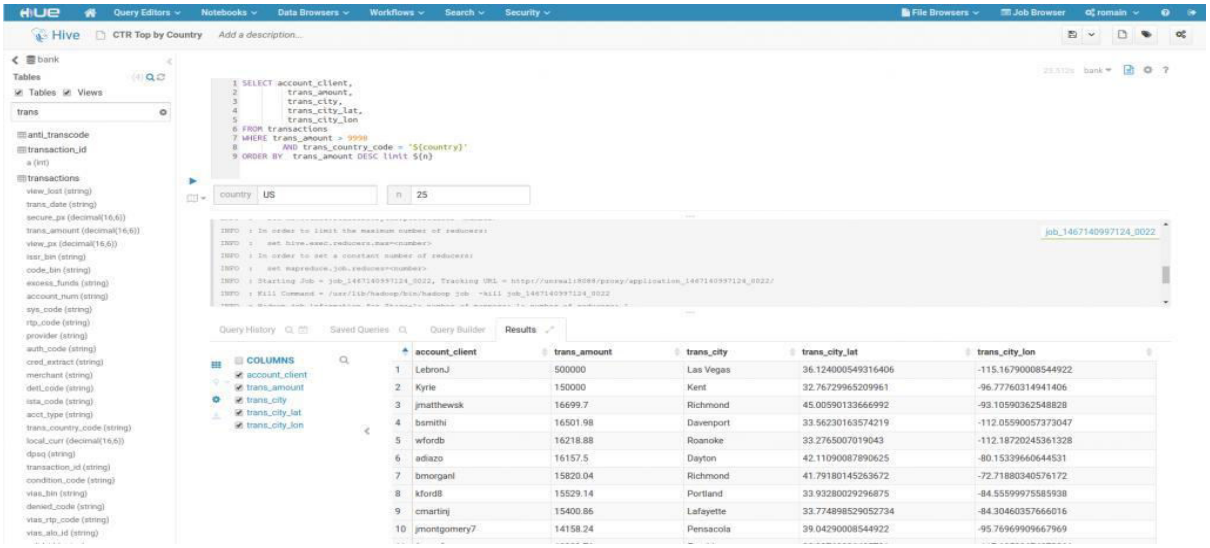
- **At Cloudera select HUE,** Login to it, (*Login and password is* ***cloudera)***



- **Observe HUE layout**, HUE provides a **query editors** that connect to Hive. Editors → select Hive from the available editor list.



- **Screen: Hive query editor** with a text area and an Execute or **Run button**.



- **at Hive editor, write Queries in, Hive Query Language (HQL)**

```
USE company_db;
CREATE TABLE sales (
order_id INT,
customer_name STRING,
product STRING,
quantity INT,
price FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

```
CREATE TABLE customers (
    customer_name STRING,
    location STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

```
LOAD DATA INPATH '/user/cloudera/sales_data/sales.csv' INTO TABLE sales;
LOAD DATA INPATH '/user/cloudera/sales_data/cust.csv' INTO TABLE customers;
```

- **Analyze the data using Hive SQL queries**

```
SELECT * FROM company_db.sales;
```

```
SHOW DATABASES;

SHOW TABLES IN company_db;

SELECT * FROM company_db.customers;

SELECT * FROM sales WHERE price > 100;

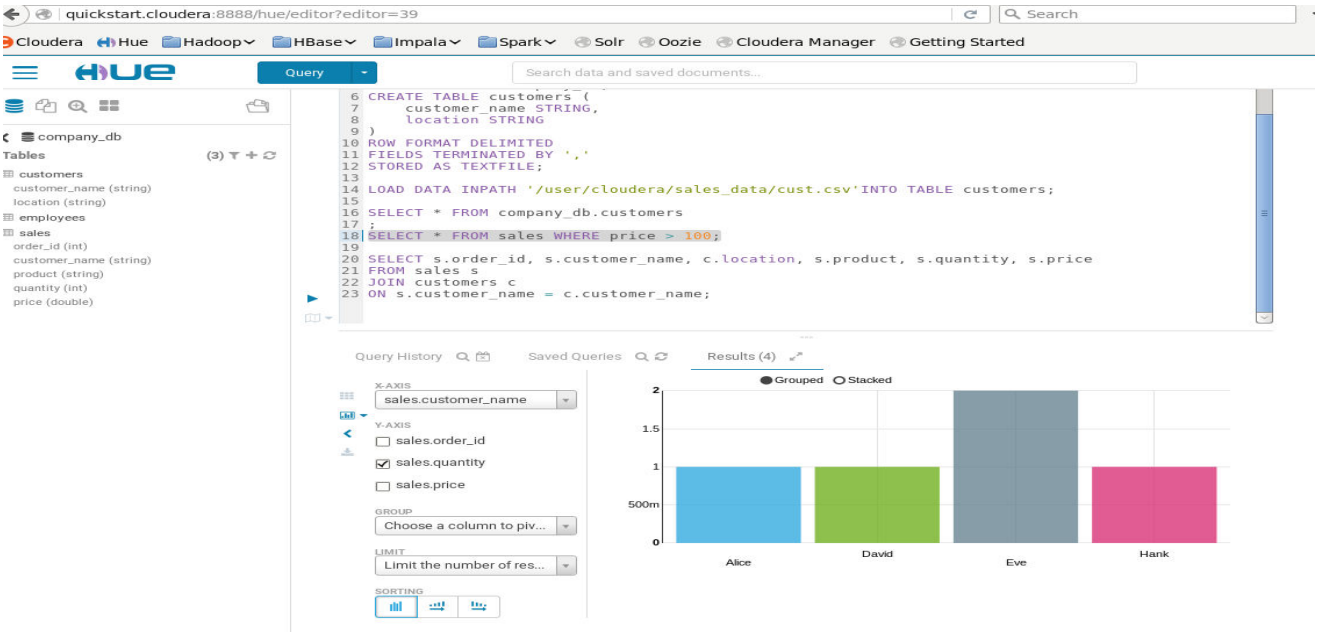
SELECT s.order_id, s.customer_name, c.location, s.product, s.quantity, s.price

FROM sales s

JOIN customers c

ON s.customer_name = c.customer_name;
```

OUTPUT:



- **to Generate a report from HUE**

HUE's query editor displays **query results in** a **tabular format**. Enabled services, may also have **visualization and dashboard** features.

Query results can be **exported to CSV or Excel**.

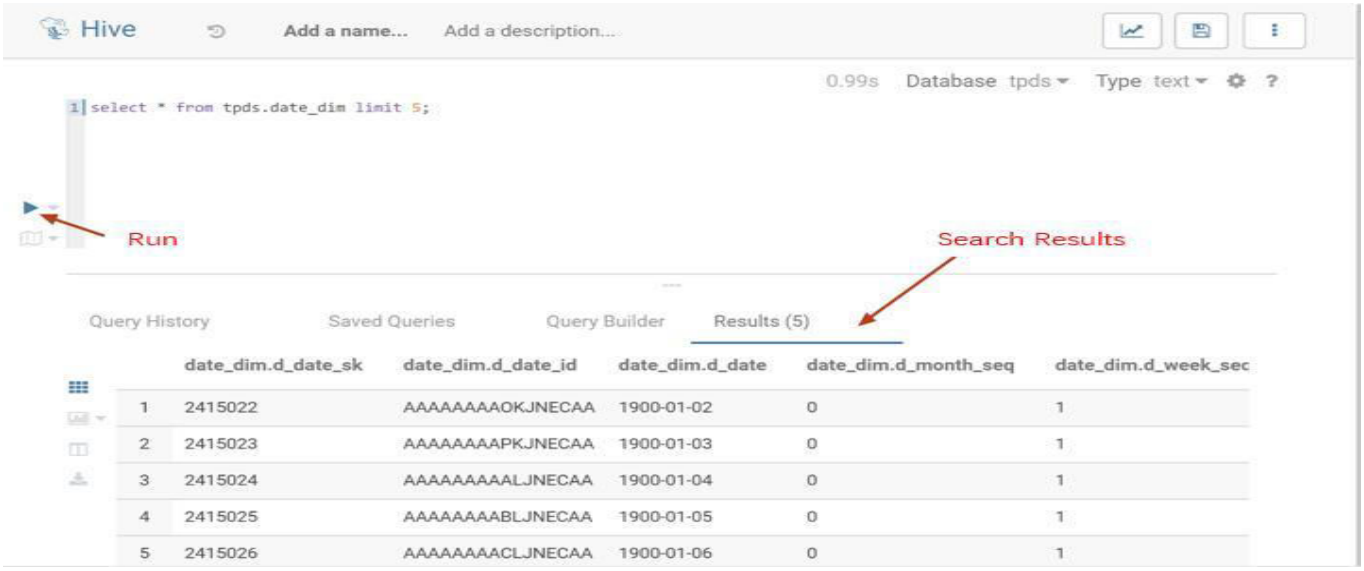
Step 1 — execute the query, Run the query. Wait for query to complete, and verify

Step 2 — Export the results

Look for the download or export option in the **results pane**.

Select CSV or Excel.

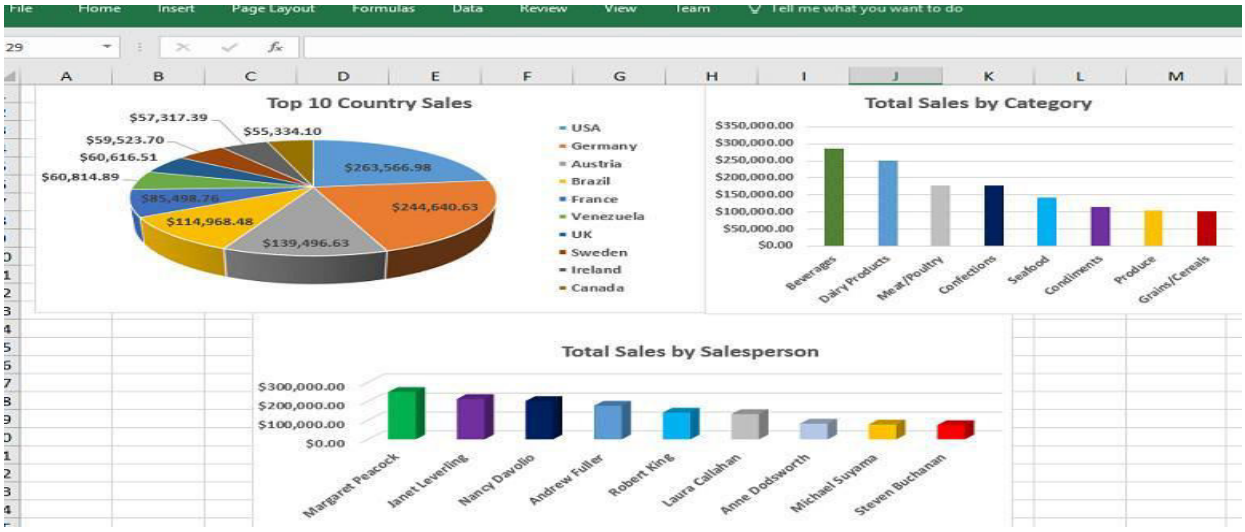
The exported file can be opened in Microsoft Excel.



	date_dim.d_date_sk	date_dim.d_date_id	date_dim.d_date	date_dim.d_month_seq	date_dim.d_week_sec
1	2415022	AAAAAAAAOKJNECAA	1900-01-02	0	1
2	2415023	AAAAAAAAPKJNECAA	1900-01-03	0	1
3	2415024	AAAAAAAAALJNECAA	1900-01-04	0	1
4	2415025	AAAAAAAABLJNECAA	1900-01-05	0	1
5	2415026	AAAAAAAACLJNECAA	1900-01-06	0	1

Step 3 — create the final report

In Excel, use the exported category to create a bar chart titled by Product Category. Include the HQL query, result table, chart, and interpretation in report.



Appendix – 8

Description of Word-count PySpark program execution in Colab

The program performs two main tasks:

1. Word Count: Counts how many times each word occurs in a text file.
2. Character Count: Selects words whose occurrence count meets a specified threshold and counts the characters in those selected words.

.1. Import the required libraries

```
import re
from pyspark.sql import SparkSession
```

- `import re`: Imports Python's regular expression module. It helps extract words while ignoring punctuation.
- `from pyspark.sql import SparkSession`: Imports `SparkSession`, which is used to start and manage a Spark application.

For example, `re.findall(r"\b\w+\b", "Hello, Spark!")` extracts `Hello` and `Spark` as words.

2. Create the Spark session

```
spark = (
    SparkSession.builder
    .master("local[*]")
    .appName("SparkWordCount")
    .getOrCreate()
)
```

Explanation of each statement:

Statement	Purpose
`SparkSession.builder`	Begins configuring the Spark session.
`.master("local[*]")`	Runs Spark locally, using available CPU cores.
`.appName("SparkWordCount")`	Gives the application the name `SparkWordCount`.
`.getOrCreate()`	Gets an existing Spark session or creates one if necessary.

Why do we need Spark?

Spark processes large datasets using distributed processing techniques. In Google Colab, `local[*]` lets us practise Spark on the notebook's available machine rather than on a separate cluster.

3. Obtain the SparkContext

```
sc = spark.sparkContext
```

`SparkContext` is the entry point for Spark's RDD-based operations.

RDD stands for Resilient Distributed Dataset. It is a collection of data that Spark can process in parallel.

We use `sc` to read the input file and perform word and character counting.

4. Set the threshold

```
threshold = 2
```

The threshold specifies the minimum number of times a word must occur to be selected for character counting.For example:

Word Occurrences Selected?

`spark` 3 Yes

big	2	Yes
fast	1	No

Because the condition is `count >= threshold`, words occurring exactly twice are also selected.

5. Read the file and split it into words

```
tokenized = sc.textFile("/content/input.txt").flatMap(
    lambda line: re.findall(r"\b\w+\b", line.lower())
)
```

This statement performs three operations.

- `sc.textFile("/content/input.txt")` reads the text file, creating an RDD of lines.
- `line.lower()` converts all letters to lowercase. Thus, `Spark` and `spark` are treated as the same word.
- `re.findall(r"\b\w+\b", line.lower())` extracts individual words, ignoring punctuation.
- `flatMap()` converts the words extracted from each line into one RDD containing individual words.

Suppose the input file contains:

```
Spark is fast
Spark is powerful
```

After tokenization, the data is conceptually:

```
spark
is
fast
spark
is
powerful
```

What is `lambda`?

A `lambda` is a small anonymous function. For example:lambda line： line.`lower()`means: take a line and return the same line in lowercase.

`flatMap()` is useful because each input line can produce several output words.

6. Count the occurrences of each word

```
wordCounts = tokenized.map(
    lambda word: (word, 1)
).reduceByKey(
    lambda v1, v2: v1 + v2
)
```

the most important parts of the program.

Step A: `map()`

```
lambda word: (word, 1)
```

Each word is converted into a key-value pair. The word is the key, and `1` represents one occurrence.

For example, the input words:

```
spark
is
fast
spark
is
```

output:

```
(spark, 1)
(is, 1)
(fast, 1)
(spark, 1)
(is, 1)
```

Step B: `reduceByKey()`

```
lambda v1, v2: v1 + v2
```

`reduceByKey()` combines values associated with the same key by adding them.

The result becomes:

```
(spark, 2)
(is, 2)
(fast, 1)
```

Word Count: How it works

Input words

spark, is, fast, spark, is

`map()`

(spark,1), (is,1), (fast,1), (spark,1), (is,1)

`reduceByKey()`

Add values belonging to the same word

Final word counts

spark: 2, is: 2, fast: 1

7. Display the word counts

```
print("Word counts:")
for word, count in sorted(wordCounts.collect()):
    print(word, count)
```

understand each part:

- `collect()` retrieves all word-count results from Spark to the driver program.
- `sorted()` arranges the results alphabetically by word.
- `for word, count in ...` extracts the word and its count from each pair.
- `print(word, count)` displays the result.

For the sample input file, the output is:

```
Word counts:
big 2
data 2
fast 1
is 3
```

Use `collect()` carefully with very large datasets because it brings all results into the driver's memory.

8. Filter words using the threshold

```
filtered = wordCounts.filter(
    lambda pair: pair[1] >= threshold
)
```

The `filter()` operation keeps only those records satisfying the specified condition.

Each record has this format:

```
(word, count)
```

Here:

- `pair[0]` is the word.
- `pair[1]` is the word's count.
- `pair[1] >= threshold` checks whether the count is at least 2.

With the sample input, the selected records are:

```
(big, 2)
(data, 2)
(is, 3)
(spark, 3)
```

Words occurring only once are removed from the subsequent character-counting stage.

9. Count the characters

```
charCounts = filtered.flatMap(
    lambda pair: list(pair[0])
).map(
    lambda c: (c, 1)
).reduceByKey(
    lambda v1, v2: v1 + v2
)
```

This statement uses same MapReduce-style logic as word counting, but processes characters instead of words.

Step A: Extract the word

```
lambda pair: list(pair[0])
```

`pair[0]` retrieves the word from a record.

For example:

```
(big, 2)   → ['b', 'i', 'g']
(data, 2)  → ['d', 'a', 't', 'a']
(is, 3)    → ['i', 's']
(spark, 3) → ['s', 'p', 'a', 'r', 'k']
```

Notice that the count in each pair is not used here. Each qualifying word contributes its characters once.

Step B: Convert characters into key-value pairs

```
.map(lambda c: (c, 1))
```

Each character is assigned a value of `1`.

For example:

```
['b', 'i', 'g'] → (b,1), (i,1), (g,1)
```

Step C: Add counts for identical characters

```
.reduceByKey(lambda v1, v2: v1 + v2)
```

Spark combines all occurrences of the same character.

For the selected words `big`, `data`, `is` and `spark`, the final character counts are:

Character	Count
a	3
b	1
d	1

For instance, `a` occurs twice in `data` and once in `spark`, so its total is 3.

10. Display the character counts

```
print("\nCharacter counts:")
for character, count in sorted(charCounts.collect()):
    print(character, count)
```

- `"\n"` inserts a blank line before the heading.
- `charCounts.collect()` retrieves the character-count results.
- `sorted()` sorts the characters alphabetically.
- The loop prints each character and its count.

The output is: `Character counts:`

```
a 3
b 1
d 1
```

11. Create the Spark session's cleanup

```
spark.stop()
```

This stops the Spark session and releases its resources after the program completes.

****

Appendix – 9

Program 9: Data Analysis and Report Generation Using CDH and HUE

1. Aim of the experiment

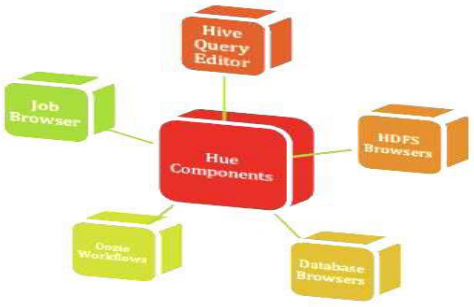
To use CDH (Cloudera Distribution for Hadoop) and HUE (Hadoop User Interface) to upload, store, analyze, and generate reports from a sample dataset using Hadoop Distributed File System (HDFS) and Hive queries.



2. Understanding the tools

CDH — Cloudera Distribution for Hadoop

CDH is a Hadoop distribution that provides components for storing and processing large datasets, such as HDFS, YARN, MapReduce, and Hive.



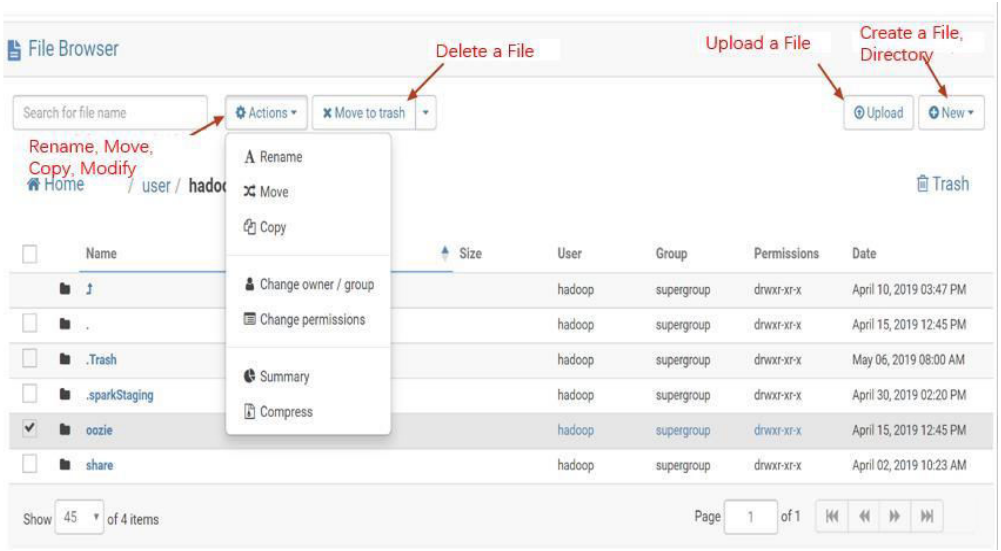
HUE — Hadoop User Interface

HUE provides a web-based interface for browsing HDFS files, exploring tables, running SQL queries through configured Hive or Impala services, and exporting query results.

10. The relationship is simple:
11. Sample dataset
12. CSV file on your computer
13. HUE File Browser
14. Upload the CSV to HDFS
15. Hive table and SQL queries
16. Analyze and summarize the data
17. Report generation
18. Query results, CSV export and report

3. Illustrative screen snapshots

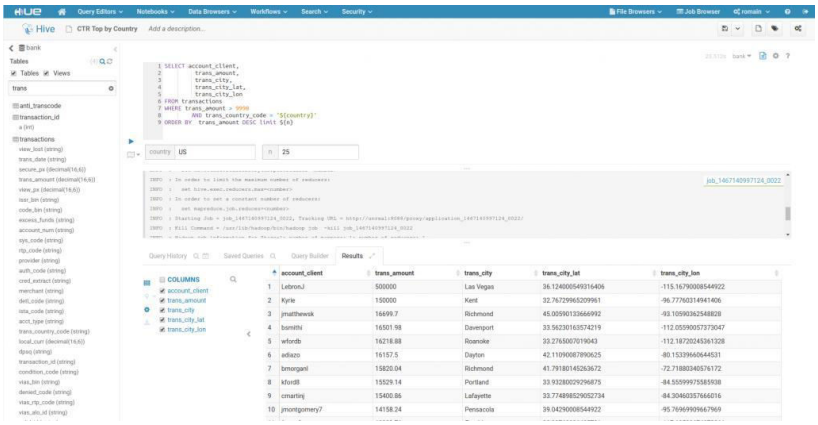
. The layout and available options may differ between CDH/HUE versions.



Screen 1 — HUE File Browser: locate HDFS directories and upload your sample file.

Screen 2 — HUE Query Editor: write Hive SQL statements and inspect the results.

The full experiment workflow is shown below.

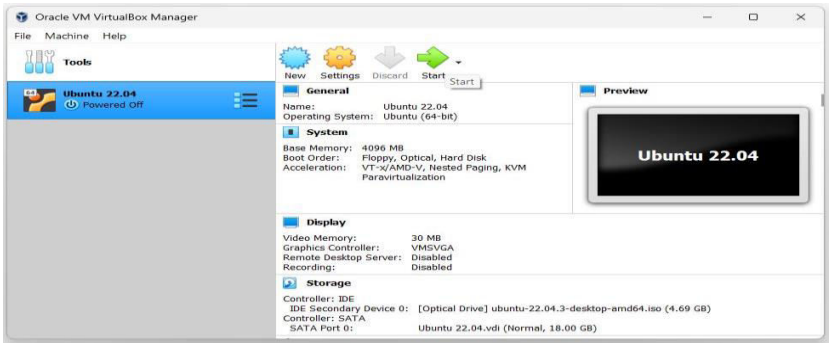


4. Software requirements and how to access HUE

To install CDH, follow the approach.

- If you still have Cloudera QuickStart VM in VirtualBox: use that environment first. You do not need to install CDH again if the VM and its services are working.

Steps to open HUE



Step 1 — Start VirtualBox

Open VirtualBox in Windows and start your Cloudera QuickStart VM. Wait for the Linux desktop to load.

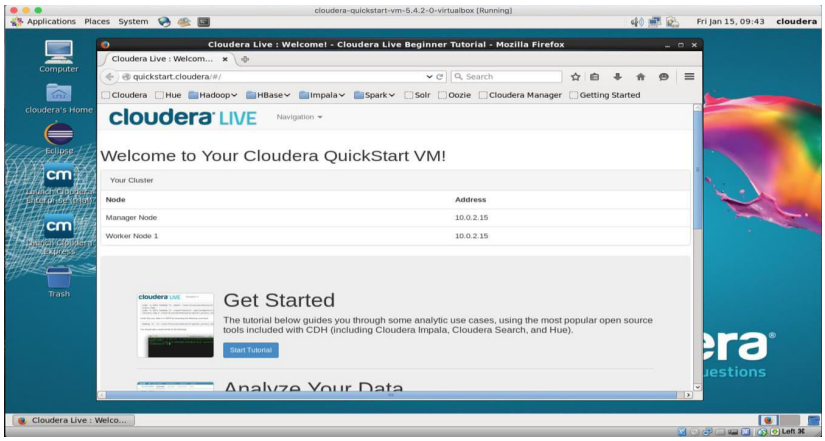
Step 2 — Check the VM

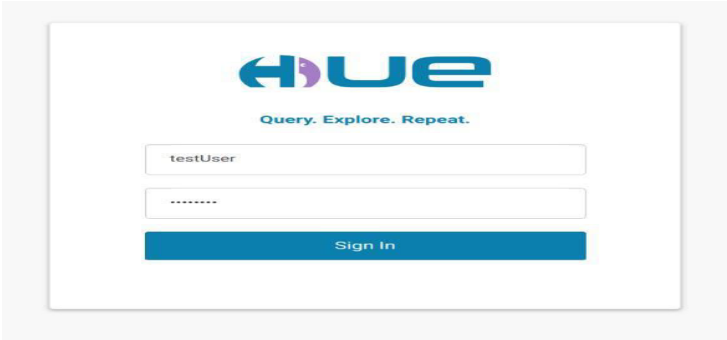
If a terminal is available, open it and enter:

hostname

hdfs dfs -ls /

If HDFS responds with directory information, that is a useful sign that HDFS is accessible.





Step 3 — Open HUE

In the VM browser, try: http://localhost:8888

If using a browser on Windows, the VM's network configuration may require its IP address instead of localhost. On a standard Cloudera QuickStart VM, the default login is often cloudera for both username and password, unless it has been changed..

5. Sample dataset for the lab

We will use a sales dataset because it allows us to demonstrate data loading, filtering, aggregation, and report generation using a small, understandable example.

Create a file named sales_data.csv on your computer with the following contents. **sales_data.csv**

10 records Copy the following into Notepad and save as sales_data.csv, selecting “All Files” as file type.

id,product,category,price,quantity,sale_date

1,Laptop,Electronics,50000,2,2026-01-05

2,Mouse,Electronics,800,5,2026-01-06

3,TShirt,Fashion,600,10,2026-01-07

4,Jeans,Fashion,1500,4,2026-01-08

5,Sofa,Home,25000,1,2026-01-09

6,Chair,Home,3000,4,2026-01-10

7,Football,Sports,900,3,2026-01-11

8,CricketBat,Sports,1800,2,2026-01-12

9,Keyboard,Electronics,2000,3,2026-01-13

10,Jacket,Fashion,2200,2,2026-01-14

The dataset contains six fields:

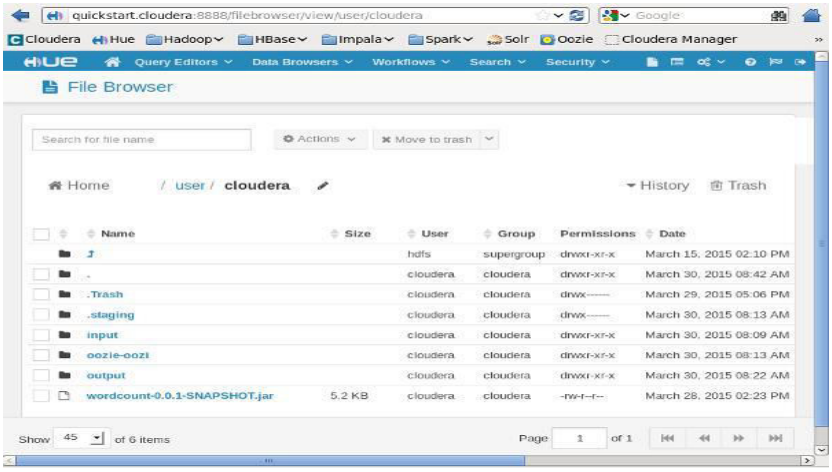
Field	Description
id	Unique transaction identifier
product	Name of the product
category	Product category
price	Price per unit
quantity	Units sold
sale_date	Date of sale

To calculate sales revenue using:

$$\text{Total Revenue} = \sum(\text{price} \times \text{quantity})\text{Total Revenue} =\ \sum(\text{price}\times\text{quantity})\text{Total Revenue}=\sum(\text{price}\times\text{quantity})$$

6. Upload the dataset to HDFS using HUE

Expected screen: HUE → File Browser or Storage Browser → HDFS.



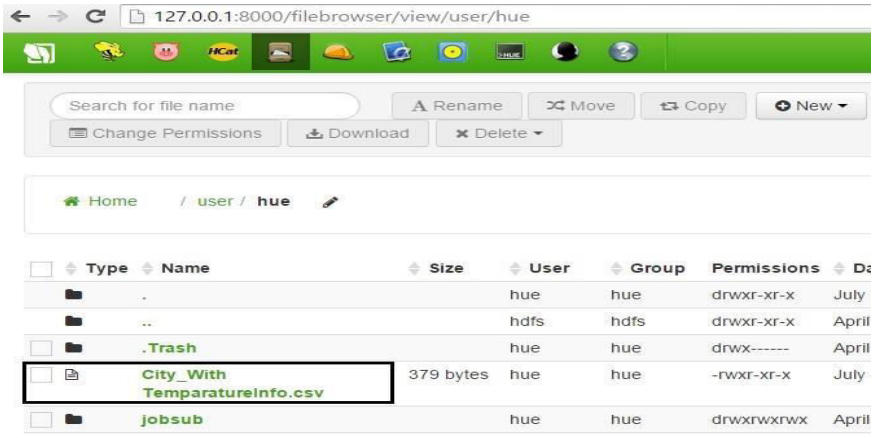
Step 1. Create the input directory

In the HUE File Browser, navigate to HDFS. Use the New or Create Directory option to create: /user/cloudera/bda_lab/sales_input



Step 2. Upload the file

Open the sales_input directory, select Upload, choose sales_data.csv, and confirm the upload.



Step 3. Verify the upload

Confirm that sales_data.csv appears inside the directory. If HUE version has no upload option, you can upload the file using the terminal instead.

Alternative: upload through the Cloudera terminal

Use these commands in the VM terminal if HUE's file browser is unavailable.

hdfs dfs -mkdir -p /user/cloudera/bda_lab/sales_input

If the CSV is in the VM user's home directory, upload it with:

hdfs dfs -put sales_data.csv /user/cloudera/bda_lab/sales_input/

Verify the file:

hdfs dfs -ls /user/cloudera/bda_lab/sales_input

note: These terminal commands require sales_data.csv to exist in the directory from which you run the command. A file saved on the Windows desktop is not automatically available inside the Linux VM.

7. Create a Hive table using HUE

HUE provides a query editor that can connect to Hive. The exact menu labels vary by version; look for Query Editors → Hive, or select Hive from the available editor list.

Expected screen: Hive query editor with a text area and an Execute or Run button.

Copy and execute the following SQL:

```
CREATE DATABASE IF NOT EXISTS bda_lab;
USE bda_lab;
CREATE EXTERNAL TABLE IF NOT EXISTS sales (
    id INT,
    product STRING,
    category STRING,
    price DOUBLE,
    quantity INT,
    sale_date STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/cloudera/bda_lab/sales_input';
```

This creates a Hive table called sales and associates it with the CSV files in the HDFS directory.

Note: The CSV has a header row. In environments that support it, add the following table property to skip that row: TBLPROPERTIES ("skip.header.line.count"="1");

If you have already created the table without this property, you can alter it:

```
ALTER TABLE bda_lab.sales
SET TBLPROPERTIES ("skip.header.line.count"="1");
```

Execute the statements one at a time if your version of HUE does not support running multiple SQL statements together.

Verify the table

Run: USE bda_lab;

SHOW TABLES;

Expected result: **tab_name**

; sales

inspect the records:

SELECT * FROM sales LIMIT 10;

If the result contains a row with id equal to id, the header has not been skipped correctly. Fix the table property before proceeding.

If you receive an error, check that Hive is available in HUE and that the table's HDFS location is correct.

8. Analyze the data using Hive SQL queries

perform the main analysis required for the BDA lab.

Query 1: Display all sales records

```
SELECT * FROM bda_lab.sales;
```

Purpose: Retrieve all records from the sales table.

output: 10 rows, with the header row excluded.

Query 2: Find total revenue for each product category

```
SELECT
    category,
    SUM(price * quantity) AS total_revenue
FROM bda_lab.sales
GROUP BY category
ORDER BY total_revenue DESC;
```

result:

Category	Total revenue
Electronics	110,000
Home	37,000
Fashion	16,400
Sports	6,300
Total	169,700

Interpretation: Electronics generates the highest revenue, while Sports generates the lowest revenue

Query 3: Find the total number of products sold in each category

```
SELECT
    category,
    SUM(quantity) AS units_sold
FROM bda_lab.sales
GROUP BY category
ORDER BY units_sold DESC;
```

Result:

Category	Units sold
Fashion	16
Electronics	10
Home	5
Sports	5

Interpretation: Fashion has the highest sales volume in terms of units sold.

Query 4: Find the highest-priced product

```
SELECT product, price
FROM bda_lab.sales
ORDER BY price DESC
```

LIMIT 1;

Result:

Product Price

Laptop 50,000

Query 5: Calculate the average product price

SELECT AVG(price) AS average_price

FROM bda_lab.sales;

Result:

average_price

8,880

Query 6: Find total revenue and transaction count

SELECT

COUNT(*) AS transaction_count,

SUM(price * quantity) AS total_revenue,

AVG(price * quantity) AS average_transaction_value

FROM bda_lab.sales;

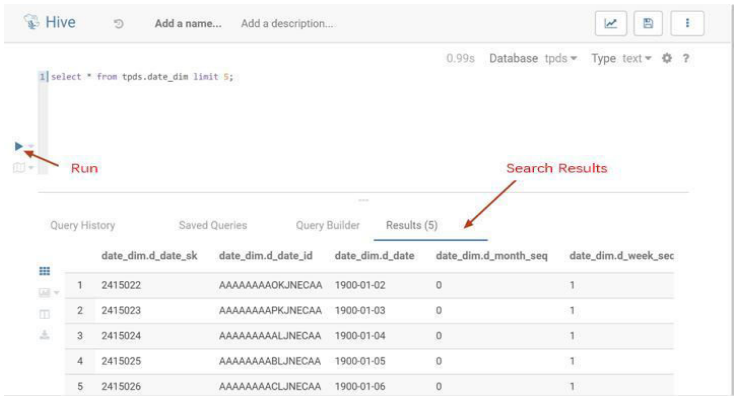
Result:

Transactions	Total revenue	Average transaction value
10	169,700	16,970

These values assume the sample CSV was loaded correctly and its header row was excluded.

9. Generate a report from HUE

HUE's query editor displays query results in a tabular format. Depending on the version and enabled services, you may also have visualization and dashboard features. Query results can be exported to CSV or Excel.



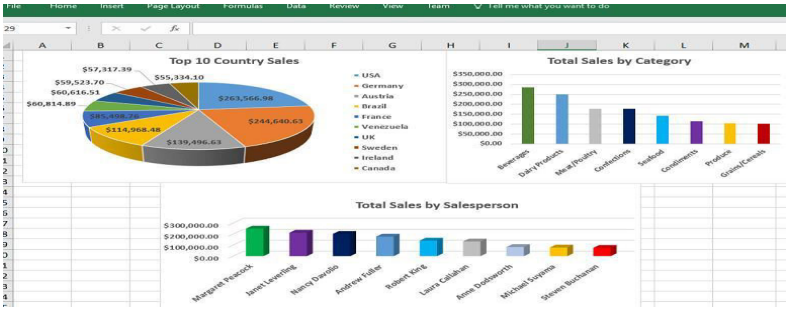
Step 1 — execute the query

Run the category revenue query from Sec 8. Wait for query to complete, and verify the four category totals.



Step 2 — Export the results

Look for the download or export option in the results pane. Select CSV or Excel if available. The exported file can be opened in Microsoft Excel.



Step 3 — create the final report

In Excel, use the exported category totals to create a bar chart titled Total Revenue by Product Category. Include the SQL query, result table, chart, and interpretation in your lab report.

Suggested report format

Sales Analysis Report

Sample report structure for the BDA practical

1. Dataset summary

10 transactions, 10 products, 4 categories.

2. Key findings

- Overall revenue: ₹1,69,700, assuming the price field represents Indian rupees.
- Highest-revenue category: Electronics.
- Highest-volume category: Fashion.
- Highest-priced product: Laptop.

3. Evidence

Include screenshots of the HDFS file, successful Hive query, query result table, and exported report or chart.

10. Troubleshooting common problems

Problem	What to check
HUE page does not open	Confirm the VM is running, then check whether the configured HUE service and port are available.
HDFS command fails	Check HDFS service status and permissions.
Hive editor is missing	Confirm Hive and its HUE connection are configured and available.
Table contains a header row	Set skip.header.line.count to 1 and rerun the query.
Table returns zero rows	Verify that the HDFS directory contains the CSV and that the table location points to the correct directory.
Query fails with a permission error	Check HDFS directory permissions and the user running Hive.
CSV export is unavailable	Copy the query result into a spreadsheet or use the Hive CLI to export the result, if available.

11. Result

The sample dataset was analyzed using Hive SQL queries through HUE. The experiment demonstrates how to upload data to HDFS, create a Hive table, perform aggregation and filtering operations, and export the results for report generation.

12. Conclusion

CDH and HUE provide an integrated environment for big-data storage, processing, and analysis. HDFS stores the dataset, Hive executes SQL-based analytical queries, and HUE provides a web interface to explore the data and inspect or export the results.